TesseraHash

A Post-Quantum-Era Content Identifier

73-byte SHAKE-256 content hash with provenance

Project 2 of 3 in the Tessera monorepo

Bahram Samani

Tessera monorepo maintainer

Whitepaper, Version 1.0

Released 2026-05-21

Build date: June 15, 2026 **Build time:** 18:12 UTC

Source: `tessera-hash/latex/main.tex`

Spec version: v1.0 **Wire variant:** `0x03`

TesseraHash is the content-hash project in the Tessera monorepo, which also contains TesseraID (variant `0x01`/`0x02`, 41-byte identifier) and TesseraSeal (variant `0x04`, ML-KEM-1024 encryption envelope). When TesseraID identifies a record whose content needs an integrity binding, use TesseraHash; when that content also needs confidentiality, wrap with TesseraSeal. See Section 9 for the cross-project relationship.

Contents

Conventions and Requirements Language **3**

TL;DR **3**

1 Threat Model **4**
1.1 What TesseraHash provides . . . 4
1.2 What TesseraHash does not provide . . . 4
1.3 Quantum-relevant attacks . . . 5
1.4 Random source quality . . . 5

2 Wire Format (Normative) **5**
2.1 Header (offsets 0–8) . . . 5
2.2 Tail (offsets 9–72) . . . 6
2.3 Length and identifier-equality semantics . . . 6

3 SHAKE-256 Binding **7**
3.1 Why SHAKE-256 over SHA3-512, SHA-512, BLAKE3, or Blake2b . . . 7
3.2 Implementation sources . . . 7
3.3 What about salts? . . . 8
3.4 Variable-length input . . . 8

4 Canonical Encodings **8**
4.1 Base64URL (RFC 4648 §5) . . . 8
4.2 Crockford Base32 (canonical form) . . . 8
4.3 Why two encodings . . . 9
4.4 Examples . . . 9

5 Generator and Provenance **9**
5.1 The stateful generator . . . 9
5.2 The stateless hash-only function . . . 10
5.3 The `hash_at` function . . . 11
5.4 What the provenance header tells you . . . 11

6 Security and Collision Math **11**
6.1 Collision and second-preimage bounds . . . 11
6.2 Why 512 bits, not 256 bits . . . 12
6.3 What an attacker controls . . . 12
6.4 Where the timestamp leaks . . . 12
6.5 Side channels . . . 13
6.6 Random source quality . . . 13

7 Conformance and Reference Implementations **13**
7.1 KAT vector files . . . 13
7.2 Cross-language runner . . . 14
7.3 Reference implementations . . . 14

7.4 Tests that must keep passing in every language 14
7.5 Benchmark methodology and reference numbers 15

8 Comparison with Existing Formats **15**
8.1 Hash functions and digest formats 15
8.2 Identifier formats with embedded hashes 16
8.3 Where TesseraHash wins 16
8.4 Where TesseraHash does not fit 17

9 The Tessera Monorepo Context **17**
9.1 Project relationship 17
9.2 Canonical patterns 17
9.3 Shared library code 18
9.4 Shared discipline 18
9.5 When *not* to use TesseraHash 19

A Test Vectors **19**

B Quick Reference **20**

C Glossary **20**

D The Tessera Monorepo **21**

E Author and Document Provenance **21**

Conventions and Requirements Language

The key words "MUST", "MUST NOT", "REQUIRED", "SHALL", "SHALL NOT", "SHOULD", "SHOULD NOT", "RECOMMENDED", "NOT RECOMMENDED", "MAY", and "OPTIONAL" in this document are to be interpreted as described in BCP 14 (Bradner, 1997; Leiba, 2017) when, and only when, they appear in all capitals, as shown here.

TL;DR

TesseraHash is a 73-byte, lexicographically-sortable **content identifier** with a 512-bit SHAKE-256 (fip, 2015) hash tail and 9 bytes of provenance metadata (48-bit millisecond timestamp, variant byte, 16-bit counter). It is *not* a hash function, *not* a MAC, *not* a signature, and *not* an encryption envelope. It is a *content identifier* that carries hash-derived collision resistance plus enough metadata to make the identifier itself sortable and traceable to a generator and time.

TesseraHash is the second of three projects in the Tessera monorepo. It shares its 9-byte header layout with TesseraID (Josefsson, 2006; Grover, 1996) (41-byte identifier, variants `0x01`/`0x02`) and TesseraSeal (ML-KEM-1024 (fip, 2024a) encryption envelope, variant `0x04`); TesseraHash owns variant byte `0x03`. Cross-project decoders reject every byte they do not own, by KAT contract.

Wire layout (73 bytes total):

```
Offset  Size  Field
------  ----  ------------------------------------------------------------
0       6     TS          (timestamp_ms, big-endian)
6       1     VARIANT     (= 0x03 for TesseraHash v1)
7       2     COUNTER     (monotonic per generator, per ms)
9       64    TAIL        (SHAKE-256(content, 64 bytes) = 512 bits)
```

Encodings: 73 bytes encode to **98 characters** of canonical Base64URL (Josefsson, 2006) (no padding) or **117 characters** of Crockford Base32 (Crockford). Both encodings round-trip to identical 73-byte binary; binary sort and Crockford ASCII sort both reflect generation time. The Crockford form preserves case-insensitive transcription and accepts the standard `I/L` → `1` and `O` → `0` confusable foldings on decode; encode emits the canonical alphabet only.

Security target: the 512-bit SHAKE-256 tail provides 2^{256} classical second-preimage and birthday-collision resistance, dropping to 2^{256} Grover-second-preimage (Grover, 1996) and $\sim 2^{170}$ Brassard–Høyer–Tapp (Brassard et al., 1998) collision under a quantum adversary — well above the 128-bit post-quantum margin that motivates TesseraID's 256-bit random tail. The timestamp and counter are *provenance metadata*, not security claims: two callers hashing the same content from different times produce identifiers that share the 64-byte tail and differ only in the 9-byte header.

At a glance

- **Content-addressed, not random.** Two TesseraHashes of the same content share the 64-byte tail; the 9-byte header gives a sortable, generator-aware view.
- **Not a hash function.** TesseraHash is a wire format *built on* SHAKE-256; it is not itself a primitive you would substitute for SHA-3 in a protocol.

- **Not a MAC, not a signature, not an envelope.** TesseraHash provides no authentication and no confidentiality; pair with TesseraSeal (section 9) when those properties are required.
- **Inherits monorepo discipline.** Spec-first LaTeX; Python is the readable reference; C pins the canonical ABI; cross-language KAT vectors gate CI.

1 Threat Model

TesseraHash is a *public artifact*: assume an attacker sees every byte of every identifier, can replay them, and may submit arbitrary content as input to a TesseraHash computation. The identifier itself carries no secret. The security goals reduce to three properties of the wire format and the SHAKE-256 (fip, 2015) tail.

1.1 What TesseraHash provides

1. **Collision resistance of the tail.** For any two distinct contents $c_1 \neq c_2$, the probability that SHAKE-256$(c_1, 64)$ = SHAKE-256$(c_2, 64)$ is bounded by the SHAKE-256 collision-resistance assumption (2^{256} classical, $\sim 2^{170}$ Brassard–Høyer–Tapp (Brassard et al., 1998) quantum).
2. **Second-preimage resistance of the tail.** Given a target h, the cost of finding any c with SHAKE-256$(c, 64) = h$ is $\min(2^d, 2^{256}) = 2^{256}$ classical per FIPS 202 (fip, 2015) (the SHAKE-256 sponge capacity caps generic preimage-class security at 256 bits regardless of output length d), and 2^{256} under Grover (Grover, 1996) on the 512-bit output — generations beyond any plausible attacker budget.
3. **Malleability resistance of the wire encoding.** The 73-byte binary and its canonical Base64URL / Crockford Base32 encodings round-trip byte-identical; canonical decoders reject non-canonical inputs (wrong length, invalid variant byte, padding-bit anomalies in Base32).

1.2 What TesseraHash does not provide

- **Authentication.** Anyone with the content can recompute the same TesseraHash-tail. TesseraHash is keyless. If the question is "did this specific party produce this hash," use a signature (PASETO (Paragon Initiative Enterprises), ML-DSA (fip, 2024b), classical Ed25519 (Josefsson and Liusvaara, 2017)) over the TesseraHash or its referenced content. TesseraID's security analysis makes the same point for identifier authentication; both sibling projects defer producer authentication to an outer envelope.
- **Confidentiality.** The 64-byte SHAKE-256 tail leaks short-content guessability: for low-entropy content, an attacker who can guess the input can verify it by recomputing the tail. This is the same property that breaks "hash the password and ship the hash" patterns. If the content is low-entropy and must remain secret, wrap it in TesseraSeal (section 9); do not rely on TesseraHash for secrecy.
- **Tamper-evidence of the identifier itself.** TesseraHash has no MAC or checksum over the full 73-byte form. A bit-flipped TesseraHash may decode to another valid TesseraHash (with a different tail). If integrity matters, the TesseraHash belongs inside a signed envelope or carried

as the AEAD AD of a TesseraSeal envelope.

- **Producer non-repudiation.** Two parties who agree on the content trivially agree on the tail. Provenance comes from the timestamp/counter *and* the outer protocol, not from the TesseraHash alone.
- **Side-channel resistance.** TesseraHash is keyless, so timing attacks on the generator are uninteresting (any timing signal an attacker recovers reveals only the content, which they may already control). When TesseraHash is used *inside* a security boundary (e.g., as part of a keyed protocol), the surrounding code's side-channel posture is the relevant one.

1.3 Quantum-relevant attacks

- **Grover's algorithm (Grover, 1996)** reduces generic search cost from 2^n to $2^{n/2}$. For TesseraHash's 512-bit tail, that is 2^{256} — coinciding with the FIPS 202 classical second-preimage cap of $\min(2^d, 2^{256})$, and astronomical either way.
- **Brassard–Høyer–Tapp (Brassard et al., 1998)** reduces collision cost from $2^{n/2}$ to $2^{n/3}$. For 512 bits, that is $\sim 2^{170}$ — comfortable but the dominant quantum bound for TesseraHash.
- **KyberSlash, PQShield power-analysis, Hertzbleed, GoFetch.** Not applicable: TesseraHash has no asymmetric crypto and no secret-dependent control flow. SHAKE-256/Keccak is a fixed-structure permutation (fip, 2015) with no input-dependent timing in stock implementations.

1.4 Random source quality

TesseraHash's tail is deterministic in the content; no CSPRNG is invoked for the tail. The 16-bit per-millisecond counter advances deterministically by 1 within a millisecond, with no randomness involved. The timestamp comes from the OS wall clock (Unix real-time clock, milliseconds since the epoch); the generator clamps on backward clock jumps so emitted timestamps never regress (section 2). TesseraID's discipline on `getrandom(2)` / `getentropy(3)` / `BCryptGenRandom` does not apply to TesseraHash because the format has no random field.

2 Wire Format (Normative)

Big-endian throughout. **Total length: 73 bytes. Fixed.**

```
Offset  Size  Field
------  ----  ------------------------------------------------------------
0       6     TS          (timestamp_ms, 48-bit big-endian unsigned)
6       1     VARIANT     (= 0x03 for TesseraHash v1)
7       2     COUNTER     (16-bit big-endian unsigned, monotonic per ms)
9       64    TAIL        (SHAKE-256(content, 64 bytes))
```

2.1 Header (offsets 0–8)

The 9-byte header is *layout-identical* to TesseraID's header (`TS(6) || VARIANT(1) || CTR(2)`). The same 9-byte parser used in TesseraID works on a TesseraHash for the timestamp, variant byte,

and counter fields without modification. A shared C header at `common/c/tessera_header.h` pins those offsets across all three Tessera projects (TesseraID, TesseraHash, TesseraSeal).

Timestamp. 48-bit unsigned millisecond Unix timestamp. The field range is $[0,\ 2^{48}-1]$ ms — 1970-01-01T00:00:00Z through approximately 10889-08-02 — and standard decoders MUST accept the full range. Separately, and non-normatively, the constant `MIN_TIMESTAMP_MS = 1_700_000_000_000` ($\approx$ 2023-11-14, matching the analogous TesseraID constant) is a sanity gate exposed via the optional `validate_min_timestamp` helper: a generator that produces a timestamp before that gate is almost certainly running with a misconfigured clock. The helper is *not* part of the wire contract; it exists for callers who want to cheaply reject obvious mis-parses at boundary entry points. Constructor-style APIs that accept an explicit timestamp (`hash_at`, section 5) MUST reject values outside $[0,\ 2^{48}-1]$ with an error rather than silently masking them to the field width.

Variant byte. `0x03` owns this project. Every other byte MUST be rejected at decode (`TESSERA_HASH_E_UNKNOWN_VA`). This applies uniformly to the binary decoder *and* to both canonical text decoders: the 98-character Base64URL and 117-character Crockford Base32 decoders MUST validate the variant byte after character-level decoding and reject any non-`0x03` value at decode time, rather than deferring the check to a separate validation step. The reserved-future range is shared monorepo-wide: TesseraID owns `0x01`/`0x02`, TesseraSeal owns `0x04`; `0x00` and `0xFF` are permanently reserved; `0x05`–`0xFE` are reserved-future. Cross-project rejection — in binary and in both text forms — is gated by the `invalid.json` KAT files in every project's `conformance/vectors/` directory.

Counter. 16-bit unsigned, big-endian. The generator increments the counter monotonically within a millisecond; on a new millisecond it resets to 0. On counter exhaustion (more than 2^{16} TesseraHashes generated in the same millisecond by a single generator) the generator MUST spin until the wall clock strictly exceeds the last emitted timestamp (`now > last_ts`), take that clock value as the new timestamp, and reset the counter to 0 — the same spin-to-next-millisecond discipline used by TesseraID's Standard generator. If the wall clock reads *earlier* than the last emitted timestamp (a backward clock jump), the generator clamps to `last_ts` and continues in the same-millisecond branch; emitted timestamps never regress. Constructor-style APIs that accept an explicit counter (`hash_at`, section 5) MUST reject values outside [0, `0xFFFF`] with an error rather than silently masking them.

2.2 Tail (offsets 9–72)

64 bytes of SHAKE-256 (fip, 2015) output computed deterministically from the input content. TesseraHash uses SHAKE-256 in its standard arbitrary-output-length mode with a fixed output length of 64 bytes (512 bits). The choice of SHAKE-256 over fixed-output SHA3-512 is deliberate: the same primitive is vendored in `common/c/keccak.{h,c}` for use by TesseraSeal's HKDF-SHA3-256 construction, so the C, C++, and Java implementations of all three projects share a single audited Keccak source.

2.3 Length and identifier-equality semantics

Two TesseraHashes are *content-equivalent* iff their 64-byte tails are identical. They are *wire-equal* iff all 73 bytes are identical. Code that deduplicates content should compare bytes `9..73`; code that needs a generator-provenance view (e.g., "which generator produced this identifier first?") compares

the full 73 bytes. The encodings (section 4) preserve both byte ranges.

3 SHAKE-256 Binding

TesseraHash invokes SHAKE-256 (fip, 2015) unchanged from FIPS 202: `SHAKE256(content, 64 bytes)` produces 512 bits of pseudorandom output deterministic in the input. There is no salt, no domain separator, no length-prefix wrapper, and no compression layer above SHAKE-256 itself. Two callers hashing identical content produce identical 64-byte tails.

3.1 Why SHAKE-256 over SHA3-512, SHA-512, BLAKE3, or Blake2b

- **Vs SHA3-512.** Both are Keccak-f[1600] sponge constructions with a 1024-bit output and 256-bit collision resistance. SHA3-512 (fip, 2015) pads with the SHA-3 domain separator `01`; SHAKE-256 pads with `1111`. We chose SHAKE-256 because (a) the SHAKE family's variable-output API is the one we want for any future TesseraHash variant that adjusts tail size, and (b) the rest of the Tessera monorepo uses SHAKE-256 (in TesseraSeal's HKDF salt construction and hybrid combiner) and we already vendor and audit it once.
- **Vs SHA-512.** SHA-512 is a Merkle–Damgård construction susceptible to length-extension attacks. TesseraHash is not used as a MAC, so length extension is not directly exploitable, but Keccak-based primitives are the consensus PQ-era choice and we have no reason to swim against the standardization current (fip, 2015).
- **Vs BLAKE3 or Blake2b.** BLAKE3 and Blake2b are faster on x86 and ARM than SHAKE-256. We accept the speed cost (section 7.5) in exchange for: (i) NIST-standardized primitive, (ii) zero new vendored crypto code (Keccak is already shared with TesseraSeal), (iii) FIPS 202 alignment for FIPS-relevant deployments. TesseraHash is not on the latency-critical path of most use cases; throughput numbers in section 7.5 are sufficient.

3.2 Implementation sources

Language	SHAKE-256 source	Constant-time / audit
Python	stdlib `hashlib.shake_256`	CPython hashlib backed by OpenSSL
C	vendored Keccak under `common/c/keccak.{h,c}`	shared with TesseraSeal; reviewed in tessera-
C++	wraps the C Keccak source	inherits C audit
Go	stdlib `golang.org/x/crypto/sha3` (`sha3.NewShake256`)	Go stdlib reviewed
Rust	RustCrypto `sha3` crate (`Shake256`)	RustCrypto reviewed
TypeScript	Node `node:crypto` (OpenSSL-backed)	Node 18+; Node-only
Java	vendored Keccak under `common/java/com/tessera/common/`	shared with TesseraSeal; reviewed
CPython ext	wraps the vendored C Keccak	inherits C audit

The cross-project sharing of Keccak under `common/` is a deliberate scope decision: TesseraHash alone could vendor its own copy, but TesseraSeal requires HMAC-SHA3-256 and HKDF-SHA3-256 for its KDF schedule, so the second consumer makes the `common/` extraction load-bearing.

3.3 What about salts?

Salts would defeat content-equivalence. A salted TesseraHash would no longer be a content identifier; it would be a randomized commitment. If your use case is "commit to content without revealing it," you want a keyed MAC (HMAC-SHAKE-256, or KMAC256), not TesseraHash. TesseraHash is the unsalted content hash by design.

3.4 Variable-length input

SHAKE-256 takes arbitrary-length input and TesseraHash passes it through unchanged. The shipped reference implementations are **one-shot today**: every public API accepts a complete in-memory buffer. The Python reference `hash_only(content)` accepts `bytes`, `bytearray`, or `memoryview`; the C reference takes a `const uint8_t *` content pointer and a `size_t` length and zero-copies into the sponge. A streaming/incremental mode for content larger than memory (absorbing chunks into the sponge so the cumulative output equals a single-shot hash of the concatenation) is a tracked backlog item on the project roadmap; it is not part of v1, and the vendored `common/` Keccak deliberately exposes no incremental absorb/squeeze API yet.

4 Canonical Encodings

73-byte binary TesseraHashes encode to two canonical text forms. Both are fixed-length and round-trip byte-identical with the binary.

4.1 Base64URL (RFC 4648 §5)

73 bytes → 98 characters. The Base64URL alphabet (Josefsson, 2006): `A-Z`, `a-z`, `0-9`, `-`, `_`; no padding (no trailing `=`). Encoding consumes 24 bits per 4 characters; 73 bytes = 584 bits = 24 full 3-byte groups (96 characters) plus 1 trailing byte (8 bits), and that trailing byte requires 2 more characters — 98 in total. The final character carries 2 bits of value plus 4 bits of zero padding that MUST be zero (so the last character is always one of `A`, `Q`, `g`, `w`). The decoder must validate exactly 98 input characters.

Decode validation. A canonical decoder:

1. Rejects input whose length is not 98 characters.
2. Rejects any character outside the URL-safe alphabet (no `+`, `/`, `=`).
3. Rejects inputs whose final character decodes to a 6-bit value with non-zero low 4 bits (equivalently: the last character must be one of `A`, `Q`, `g`, `w` — the four 6-bit values whose low 4 bits are zero).

The third check is the padding-bit guard. Without it, two distinct 98-character strings would decode to the same 73-byte binary, which violates canonical round-trip. The KAT file `tessera_hash_decode_b64.json` pins this rejection.

4.2 Crockford Base32 (canonical form)

73 bytes → 117 characters. The Crockford Base32 alphabet (Crockford): `0-9` then `A-Z` minus `I/L/O/U`. Encoding consumes 5 bits per character; 73 bytes = 584 bits = 116 full quintets plus 4

bits, so 117 characters carry the 4-bit excess in the final character with 1 bit of zero padding.

Decode validation.

1. Length exactly 117 characters.
2. Case-insensitive: lower-case folds to upper-case before lookup.
3. Confusable canonicalization: `I/L` → `1`, `O` → `0` accepted on decode. The encoder emits canonical glyphs only.
4. Padding bit: the final character's 5-bit value MUST have its low bit zero. Equivalent to the Base64URL padding-bit guard but in the Crockford alphabet.

Sort property. Crockford Base32 ASCII sort preserves the byte order of the underlying 73-byte binary, which preserves generation-time order (because the leading 6 bytes are the big-endian timestamp). Base64URL ASCII sort *does not* preserve byte order — its alphabet ordering does not match ASCII codepoint order. Consumers that need a textual column to sort by generation time should use the Crockford form or compare the underlying bytes.

4.3 Why two encodings

- **Base64URL** is the right choice for JSON values, HTTP headers, query strings, and database columns whose ordering does not matter. It is denser (98 chars vs 117) and is the canonical web encoding.
- **Crockford Base32** is the right choice for log lines, CLI tools, human transcription (the alphabet excludes the I/L/O/U confusables on purpose), and any database column where lexicographic sort must reflect generation time.

Both choices mirror TesseraID's encoding rationale exactly; the two projects use the same encoder code paths in every language (with different alphabet sizes per project).

4.4 Examples

- Binary TesseraHash of the empty string `b""` at timestamp 1700000000000 and counter 0: 73-byte binary → 98-char Base64URL → 117-char Crockford. Test vectors with byte-exact expected outputs ship in `tessera-hash/conformance/vectors/tessera_hash_encode_b64.json` and `tessera_hash_encode_crockford.json`.
- Decoder round-trip: any 73-byte binary b satisfies `decode_b64`(`encode_b64`(b)) $= b$ and `decode_b32`(`encode_b32`(b)) $= b$.

5 Generator and Provenance

TesseraHash ships two API shapes: a *generator* that maintains a counter for provenance-marking the same content multiple times, and a *stateless* hash-only function for callers who do not need the 9-byte header.

5.1 The stateful generator

```
g = tessera_hash.Generator()
id1 = g.new(content)              # 73 bytes: TS_now || 0x03 || ctr ||
    SHAKE256(content)
id2 = g.new(content)              # same content, fresh TS+counter
assert id1[9:] == id2[9:]         # tails identical (same content)
assert id1 != id2                 # headers differ (different TS or counter)
```

The generator is **internally locked where the language has shared-memory threading**: the `(last_ts, counter)` read-modify-write is atomic across threads (Python `threading.Lock`, Go mutex, Rust `Mutex`, Java `synchronized`). The C struct is the documented exception: it is *caller-synchronized* by contract — `tessera_hash.h` states the struct "exposes no synchronization" and callers MUST declare one generator per thread or wrap it with a mutex; the C++ wrapper inherits that contract. This matches TesseraID's `HLCGenerator` discipline (TesseraID's `StandardGenerator` is deliberately *not* locked; TesseraHash's generator is locked wherever the language can lock it, because counter monotonicity is the only provenance property the format offers and racing it produces undefined ordering).

The counter advances monotonically within a millisecond and resets to 0 on a new millisecond. On counter exhaustion (more than $2^{16} = 65{,}536$ identifiers generated by a single generator within one millisecond) the generator MUST spin until the wall clock strictly exceeds the last emitted timestamp (`now > last_ts`), take that clock value as the new timestamp, and reset the counter to 0. If the wall clock reads earlier than `last_ts` (a backward clock jump), the generator clamps to `last_ts` so emitted timestamps never regress; the spin condition `now <= last_ts` therefore also covers regression — the loop cannot exit while the clock still reads at or before the last emitted timestamp. This is the same spin-to-next-millisecond overflow discipline used by TesseraID's Standard generator (which spins on exhaustion; the "bump the logical clock by 1" rule belongs to TesseraID's *HLC* variant, not to this format).

5.2 The stateless hash-only function

```
tail = tessera_hash.hash_only(content)          # 64 bytes: SHAKE256(content, 64)
```

`hash_only(content)` returns just the 64-byte tail. There is no header, no timestamp, no counter. Two callers in different processes who hash the same content with this function get the same 64 bytes. Use this when:

- You want to deduplicate by content without committing to a generator identity.
- You want to use the tail as an AEAD AD (e.g., binding a TesseraSeal envelope to specific content; see section 9).
- You are computing the tail for verification: given an existing TesseraHash, check that the trailing 64 bytes equal `hash_only(content)` for the candidate content.

The `verify(raw, content)` function does exactly this verification: it parses the 73-byte raw form, extracts the trailing 64 bytes, recomputes `hash_only(content)`, and returns a boolean. `verify` is the canonical way to ask "does this TesseraHash reference this content?"

5.3 The `hash_at` function

For deterministic test-vector generation and KAT reproducibility:

```
raw = tessera_hash.hash_at(content, timestamp_ms=1700000000000, counter=0)
```

This bypasses the wall clock and the generator's counter state, producing a 73-byte TesseraHash whose header carries exactly the provided timestamp and counter. `hash_at` MUST reject a `timestamp_ms` outside $[0,\ 2^{48}-1]$ or a `counter` outside [0, `0xFFFF`] with an error — never by silently masking the value to the field width (Python raises `ValueError`; C returns `TESSERA_HASH_E_RANGE`; Go returns `ErrTesseraHashRange`; the other languages raise their equivalent range error). Used by `tessera-hash/conformance/scripts/generate_tessera_hash.py` to produce the KAT files (section 7).

5.4 What the provenance header tells you

- **When** (timestamp): the millisecond a particular generator first produced this identifier. Two generators that independently hash the same content at different times will produce different timestamps but identical tails. The timestamp is a fact about the *identifier*, not the content.
- **In what order** (counter): the relative ordering of identifiers from a *single* generator within a millisecond. Counters across generators are unrelated; a counter value of 7 from one generator means nothing about a counter value of 7 from another generator.
- **Which content** (tail): the 512-bit SHAKE-256 hash of the input. This is the only field that carries content identity.

What the provenance header does *not* tell you: who generated the identifier (no producer field; no signature), where it was generated (no node ID), or whether the generator's clock is honest (no clock attestation). Provenance is a self-claim of the generator; trust comes from the outer protocol.

6 Security and Collision Math

TesseraHash's security claim reduces to "SHAKE-256 is collision-resistant." This section quantifies the margin and lays out the threat boundaries explicitly.

6.1 Collision and second-preimage bounds

For an *ideal* n-bit hash output, the generic bounds are 2^n classical second-preimage, $2^{n/2}$ classical collision, $2^{n/2}$ under Grover (Grover, 1996), and $2^{n/3}$ under BHT (Brassard et al., 1998). SHAKE-256 is not an ideal d-bit hash for large d: FIPS 202 (fip, 2015) caps its generic preimage-class security at $\min(d, 256)$ bits (the sponge capacity bound) and collision at $\min(d/2, 256)$ bits. For TesseraHash's $d = 512$ output, both caps bind at 256 bits:

Property	Ideal	Grover	BHT	TesseraHash ($d = 512$)
Second preimage	2^n	$2^{n/2}$	—	$\min(2^d, 2^{256}) = 2^{256}$; Grover 2^{256}
Collision	$2^{n/2}$	—	$2^{n/3}$	2^{256}; BHT $\sim 2^{170}$

The dominant quantum bound is Brassard–Høyer–Tapp collision at $\sim 2^{170}$. To put 2^{170} in perspective: at a generation rate of 10^{12} TesseraHashes per second — vastly beyond any realistic system — finding a collision would take $\sim 10^{31}$ years, or roughly 10^{21} times the current age of the universe.

The relevant point for content-identifier use cases is that *second-preimage* is usually the threat that matters ("can an attacker produce content that hashes to this specific identifier?") and second-preimage is bounded by 2^{256} even under Grover. This is the same security margin NIST targets for ML-KEM-1024 (fip, 2024a) and AES-256-GCM-equivalent symmetric strength.

6.2 Why 512 bits, not 256 bits

A 256-bit tail (SHAKE-256 truncated, or SHA3-256) would provide 2^{128} classical collision and $\sim 2^{85}$ BHT collision. The 2^{85} figure — "one trillion years at one trillion generations per second" — is comfortable but not 30–50 year-comfortable. TesseraHash is intended to outlive any individual deployment; budgeting collision resistance for two centuries beyond the most aggressive realistic generation rate motivates the 512-bit choice. The 9 extra bytes of header relative to a hypothetical 32-byte (256-bit-tail) variant cost essentially nothing in practice.

This contrasts with TesseraID: TesseraID's 256-bit tail is sized for *unguessability* (a random field where Grover second-preimage at 2^{128} is the binding bound), not collision. The two projects use different bit budgets because the threats differ.

6.3 What an attacker controls

- **Content.** Yes — the attacker can submit any content they want. The collision/second-preimage cost is what it is.
- **Timestamp.** The attacker controls the timestamp on their own generator. The timestamp is provenance metadata; it cannot be forged "against" a third party because the third party never signed it.
- **Counter.** Similarly attacker-controlled on their own generator.
- **Variant byte.** Setting variant to anything other than `0x03` causes decode rejection. Setting variant to `0x01`, `0x02`, or `0x04` additionally triggers cross-project rejection in TesseraID's and TesseraSeal's decoders — attackers cannot smuggle a TesseraHash past a TesseraID parser by flipping the variant byte (and vice versa).
- **Tail.** The attacker chooses the content; the tail is then forced. The attacker cannot independently set the tail.

6.4 Where the timestamp leaks

The 48-bit timestamp is plaintext. An attacker who sees a TesseraHash learns approximately when the identifier was generated — not when the content was created (different generators can hash the same content at different times). If the timestamp is sensitive, encrypt the entire TesseraHash (use TesseraSeal; section 9), or do not generate the TesseraHash until you are ready to publish.

6.5 Side channels

TesseraHash has no secret-dependent control flow: the SHAKE-256 sponge processes content with input-independent permutations (fip, 2015), the counter advances deterministically, the timestamp is read from the OS clock. An attacker who can time a TesseraHash computation recovers, at most, the length of the input content (which they may control or already know).

Because TesseraHash is keyless, the side-channel disclaimer language that TesseraSeal requires (constant-time vs constant-power, KyberSlash regression, etc.) does not apply. The security guarantees rest on the cryptographic strength of SHAKE-256 (fip, 2015), not on the implementation's timing behavior.

6.6 Random source quality

TesseraHash has no random field. The CSPRNG discipline that binds TesseraID and TesseraSeal (OS `getrandom`/`getentropy`/`BCryptGenRandom` only, no userspace PRNG) is not relevant for TesseraHash: the tail is deterministic in content, the counter advances by 1, the timestamp is wall-clock.

7 Conformance and Reference Implementations

TesseraHash ships byte-identical reference implementations in eight languages, all consumed by the same cross-language KAT harness used by TesseraID and TesseraSeal. The wire format is the contract; the per-language code is a reference.

7.1 KAT vector files

`tessera-hash/conformance/vectors/`:

- `tessera_hash_decode.json` — 73-byte hex → {variant, timestamp_ms, counter, tail_hex}.
- `tessera_hash_hash_only.json` — known content → known 64-byte SHAKE-256 tail.
- `tessera_hash_encode_b64.json`, `tessera_hash_decode_b64.json` — 98-char Base64URL round-trip.
- `tessera_hash_encode_crockford.json`, `tessera_hash_decode_crockford.json` — 117-char Crockford round-trip.
- `tessera_hash_verify.json` — positive (matching content) and negative (one-bit content flip) cases for `verify(raw, content)`.
- `tessera_hash_invalid.json` — malformed inputs: wrong length, invalid variant byte in binary form (incl. cross-project `0x01`/`0x02`/`0x04` and mid-range reserved bytes), wrong-variant *text* forms (Base64URL and Crockford strings carrying non-`0x03` variants, pinning decode-time variant validation), non-canonical Base64URL/Crockford characters, padding-bit anomalies.

Vectors are generated against the Python reference (`tessera-hash/python/tessera_hash.py`) by `tessera-hash/conformance/scripts/generate_tessera_hash.py`; the generator is deterministic given fixed inputs. Shipped vectors are frozen append-only: regeneration may add records or files but MUST NOT change existing record bytes.

7.2 Cross-language runner

`tools/conformance/run.sh` discovers per-language runners under `tools/conformance/` (one directory per language: `python`, `c`, `cpp`, `go`, `rust`, `typescript`, `java`, `cpython`) and executes each against the KAT files for the projects that language implements. All eight languages consume the `tessera-id` and `tessera-hash` vector directories — every TesseraHash vector runs in all eight. Only the four languages with their own `tessera-seal` runners (Python, C, Go, Rust) additionally consume `tessera-seal/conformance/vectors/`; C++, TypeScript, Java, and the CPython extension skip that directory until their TesseraSeal ports land — there is no Python-runner fallback. Exit code 0 means every runner passed every vector it ran. The harness runs on every pull request and push (`.github/workflows/conformance.yml`) and its cross-language job is a required branch-protection check on `main`.

```
bash tools/conformance/run.sh
TESSERA_CONFORMANCE_SKIP="cpython java cpp typescript" bash
    tools/conformance/run.sh   # quick loop
```

7.3 Reference implementations

Language	Status	SHAKE-256 source	Notes
Python	complete	stdlib `hashlib.shake_256`	Readable reference (`python/tessera_hash.py`)
C	complete	vendored `common/c/keccak.{h,c}`	Canonical ABI; pins offsets and error codes
C++	complete	wraps the C library	Header-only `tessera_hash.hpp`
Go	complete	`golang.org/x/crypto/sha3`	`tessera_hash.go`
Rust	complete	RustCrypto `sha3` crate	`lib.rs`; const-time docs from RustCrypto
TypeScript	complete	`node:crypto` (OpenSSL)	`tessera_hash.ts`; Node 18+; Node-only
Java	complete	vendored `common/java` Keccak	Codec + `Generator`; 17 tests (`make test`)
CPython ext	planned	wraps vendored C Keccak	Mirrors the Python API; v1.x backlog

7.4 Tests that must keep passing in every language

- SHAKE-256 KAT vectors (canonical Keccak test vectors): hash of known inputs produces known outputs.
- Hash → Base64URL round-trip: exact 98-character length, no padding.
- Hash → Crockford Base32 round-trip: exact 117-character length, confusable folding on decode.
- Invalid-variant decode rejection: `0x00`, `0x01`, `0x02`, `0x04`, and `0x05`–`0xFF` are all rejected with `UNKNOWN_VARIANT`. Rejection applies in binary form *and* through both text decoders (98-char Base64URL, 117-char Crockford).
- `hash_at` range rejection: an out-of-range `timestamp_ms` (above $2^{48} - 1$) or `counter` (above `0xFFFF`) MUST produce an error, never a silently masked value.
- Cross-language conformance: a TesseraHash produced in one language MUST decode byte-identical in every other.
- Padding-bit guards: Base64URL final-character low-4-bits zero; Crockford final-character low-bit zero.

- Counter monotonicity within a millisecond, including spin-to-next-millisecond on counter exhaustion and clamping on backward clock jumps.

7.5 Benchmark methodology and reference numbers

Benchmarks live under `tessera-hash/benchmarks/`. Coverage is deliberately minimal today (a Python + C harness; the remaining languages are backlog — this paper does not claim per-language benchmark files exist). The harness measures four operations:

1. `hash_only(content)` on a fixed 1 KiB content buffer.
2. `Generator.new(content)` including header construction.
3. `decode(raw)` on a fixed 73-byte input.
4. `verify(raw, content)` on matching content.

TesseraHash throughput is dominated by SHAKE-256 sponge cost; the wire-format work (parsing, counter advance, Base64URL/Crockford coding) is negligible relative to Keccak. This paper intentionally publishes no point-in-time throughput numbers: consult the checked-in run outputs and methodology notes in `tessera-hash/benchmarks/` for measured figures, and treat any figure not accompanied by a checked-in run output as unverified.

8 Comparison with Existing Formats

TesseraHash occupies a specific niche: a *content identifier* that carries provenance metadata alongside a hash strong enough to outlive deployments. This section places it against adjacent constructs.

8.1 Hash functions and digest formats

Construct	Output bits	PQ posture	Provenance metadata
SHA-256	256	2^{128} classical / 2^{85} BHT collision	none
SHA3-512 (fip, 2015)	512	2^{256} classical / 2^{170} BHT collision	none
SHAKE-256 (fip, 2015)	arbitrary	up to 2^{170} BHT at 512-bit output	none
BLAKE3	256 (default)	2^{128} classical / 2^{85} BHT collision	none
BLAKE2b-512	512	2^{256} classical / 2^{170} BHT collision	none
Git SHA-1	160	2^{80} classical / $\sim 2^{53}$ BHT collision	none (broken in 2017)
Git SHA-256 (transition)	256	2^{128} classical / 2^{85} BHT collision	none
IPFS CIDv1 (multihash)	variable	depends on chosen hash	multihash codec ID
TesseraHash v1	**512**	**2^{256} classical / 2^{170} BHT collision**	**TS (48b) + CTR (16b) + VAR (8b)**

TesseraHash is not in the same category as "a hash function"; it is a wire format that bundles

SHAKE-256 with provenance. The right way to think about TesseraHash is "IPFS CIDv1 if CIDv1 had committed to one hash and one provenance shape and skipped the multihash codec table."

8.2 Identifier formats with embedded hashes

Format	Size (bytes)	Hash bits	Notes
Git commit hash (SHA-1)	20	160	Broken; transition to SHA-256 underway
Git commit hash (SHA-256)	32	256	Forward-compat; no provenance metadata
IPFS CID (default)	36	256	SHA-256 + codec + version; provenance via outer protocol
Docker image digest (SHA-256)	32	256	No provenance
Subresource Integrity (SHA-512)	64	512	Inline data-URI; no provenance
ULID (Feerasta, 2016)	16	0 (random)	Identifier, not content-bound
TesseraID	41	0 (256-bit random)	Identifier, not content-bound
TesseraHash v1	**73**	**512**	Content-bound + provenance metadata

The deliberate scope difference: TesseraID answers "which record is this?" (random; unguessable); TesseraHash answers "which content is this?" (deterministic in content; collision-resistant). When both questions matter, use both — the canonical TesseraID-as-AD pattern in TesseraSeal (section 9) lets a TesseraSeal envelope bind one TesseraID record to one TesseraHash content commitment, and the wire formats stack cleanly.

8.3 Where TesseraHash wins

- **Multi-decade collision margin.** 512-bit SHAKE-256 is the highest commonly-deployed standardized hash. 2^{256} classical collision doubles the security exponent of what SHA-256-based formats (Git, Docker, IPFS-default) provide ($2^{128} \to 2^{256}$ — a 2^{128}-fold, roughly $10^{38}\times$, increase in attack cost, not a small constant factor).
- **Provenance built in.** The 9-byte header carries enough metadata to make a TesseraHash sortable by generation time, traceable to a single generator's counter sequence, and decode-rejectable from cross-project siblings. Bare hash digests carry none of this.
- **One primitive across the monorepo.** The same SHAKE-256 implementation that produces TesseraHash tails is reused inside TesseraSeal's HKDF schedule (section 9); two primitives audited, one used everywhere.
- **Canonical text encodings.** 98-char Base64URL and 117-char Crockford Base32 with explicit padding-bit guards. The same 73 bytes in bare hex — the default presentation for SHA-512-class digests in most ecosystems — would be 146 characters; the Base64URL form is ≈33% shorter and the Crockford form ≈20% shorter, and the Crockford form sorts by generation time.

8.4 Where TesseraHash does not fit

- **If you need an unkeyed-MAC–like property.** Use HMAC-SHA3-256 or KMAC256. TesseraHash provides no producer authentication.
- **If you need to substitute for SHA-256 inside an existing protocol.** The protocol expects 32 bytes; TesseraHash emits 73. Use `hash_only(content)` (64-byte tail only) and drop the provenance.
- **If you need multihash agility** (IPFS-style). TesseraHash commits to SHAKE-256; adding a second hash requires a new variant byte under the Tessera discipline. TesseraHash will not chase the multihash codec table.
- **If you need confidentiality of the content.** Use TesseraSeal (section 9); TesseraHash reveals the content to anyone who can guess it.

9 The Tessera Monorepo Context

TesseraHash is the second project under the Tessera monorepo. Each project is independent on the wire and standalone in its API; they share variant-byte namespace, header layout, and conformance harness.

9.1 Project relationship

Project	Concern	Variant byte	Wire size (bytes)
TesseraID	Post-quantum-era identifier	`0x01` (Standard), `0x02` (HLC)	41
TesseraHash	SHAKE-256 content hash	`0x03`	73
TesseraSeal	ML-KEM-1024 encryption envelope	`0x04`	variable (header 16 + body)

The first 9 bytes of every project's wire format are layout-compatible: `TS(6) || VARIANT(1) || CTR(2)`. TesseraHash extends this with a 64-byte tail. TesseraSeal extends with a different 7-byte suffix (SUITE + KEM_CT_LEN + AD_LEN) followed by a variable body. Decoders in each project reject every variant byte they do not own; the cross-rejection contract is gated by `invalid.json` KAT files in every project.

9.2 Canonical patterns

9.2.1 TesseraHash-as-AD: bind a TesseraSeal envelope to content

When encrypting a payload, pass the TesseraHash of the cleartext as the AEAD AD. The TesseraSeal envelope is then cryptographically bound to "this specific content," not just "some content this recipient can decrypt."

```
content_hash = tessera_hash.hash_only(plaintext)          # 64 bytes (or pass full 73B raw)
envelope     = tessera_seal.encrypt(plaintext, recipient_pub,
                                    associated_data=content_hash)
# ... after delivery ...
plaintext    = tessera_seal.decrypt(envelope, recipient_priv,
                                    expected_ad=content_hash)
```

If an attacker substitutes the envelope with one carrying different content, the AEAD AD mismatch causes decryption to fail. The same pattern works with a TesseraID as AD (the canonical pattern documented in the TesseraSeal whitepaper); the choice between TesseraID-as-AD and TesseraHash-as-AD depends on whether the binding is to "a specific record" (TesseraID) or "specific content bytes" (TesseraHash). Both choices cost zero additional cryptography.

9.2.2 TesseraID + TesseraHash together: identifier-plus-content-commitment

A record-keeping system that wants *both* stable identification and content integrity can carry a TesseraID (for the record's identity over time — multiple revisions of the same record share a TesseraID) and a TesseraHash (for the current content). Updates to the content roll the TesseraHash but keep the TesseraID. Subscribers verify both: the TesseraID pins identity, the TesseraHash pins content.

9.2.3 TesseraHash of a TesseraSeal envelope

The output of a TesseraSeal encryption is a deterministic-on-input-and-randomness byte string. Computing TesseraHash over that envelope gives a content-bound identifier for the encrypted blob. This is the right pattern for an object-storage system that wants to index by ciphertext.

9.3 Shared library code

Cross-project code lives under `common/` only if more than one project depends on it today:

- `common/c/keccak.{h,c}`, `common/java/com/tessera/common/Keccak.java`: SHAKE-256/Keccak. TesseraHash uses it for the tail; TesseraSeal uses it for HKDF-SHA3-256 in its KDF schedule and for the hybrid suite's SHAKE-256 combiner.
- `common/c/tessera_header.h`: shared offset constants for the 9-byte header layout. All three projects parse the same first nine bytes the same way.
- `common/c/random.{h,c}`: OS CSPRNG wrapper. Used by TesseraID (random tail) and TesseraSeal (ML-KEM `Encaps` randomness, X25519 ephemeral keygen). TesseraHash has no random field and does not call this.
- `common/c/hkdf_sha3.{h,c}`: HMAC-SHA3-256 and HKDF-SHA3-256. TesseraSeal-only today, vendored alongside the Keccak source.

Python and Go use stdlib for SHAKE-256 (`hashlib.shake_256`, `golang.org/x/crypto/sha3`); Rust uses the `sha3` crate; TypeScript uses Node's built-in `node:crypto` (OpenSSL-backed, Node 18+, Node-only); C, C++, and Java pull from `common/`. Never re-vendor a second Keccak inside a project that already has `common/` on its include path.

9.4 Shared discipline

- One specification-first methodology: whitepaper LaTeX is normative, Python is the readable reference, C pins the canonical ABI, every other language ports byte-identical.
- One cross-language KAT harness at `tools/conformance/run.sh`; the branch-protection gate on `main`.

- One shared variant-byte namespace; cross-project rejection KATs in every project.
- One author, one set of memory invariants, one set of CLAUDE.md files coordinating PRs across the three projects.

9.5 When *not* to use TesseraHash

- For random-looking unique identifiers, use TesseraID. TesseraHash is content-bound; two callers hashing the same content produce identifiers with identical tails, which is correct for content-addressing but wrong for "a freshly-minted unique handle."
- For password hashing, use Argon2id (Biryukov et al., 2021) or scrypt. TesseraHash has no work factor and no salt; it is a fast keyless hash, not a slow-by-design password derivation function.
- For message authentication, use HMAC-SHA3-256 or KMAC256. TesseraHash is keyless.
- For encryption, use TesseraSeal. TesseraHash provides no confidentiality.

A Test Vectors

Vector 1: empty content, deterministic header

- **Content:** `b""` (zero bytes)
- **Timestamp (ms):** `1700000000000`
- **Counter:** `0`
- **Tail (SHAKE-256(b"", 64 bytes)):** the canonical FIPS 202 (fip, 2015) SHAKE-256 output for the empty string, truncated to 64 bytes.

The 73-byte binary is `TS(6, big-endian) || 0x03 || 0x00 0x00 || tail(64)`. The exact hex bytes for this vector live in `tessera-hash/conformance/vectors/tessera_hash_hash_only.json`.

Vector 2: short content

- **Content:** `b"abc"`
- **Tail:** SHAKE-256(`b"abc"`, 64 bytes), canonical FIPS 202 output.

Used by the round-trip tests in `tessera_hash_verify.json`: a one-bit flip anywhere in the content changes the tail in many bit positions (avalanche property of Keccak-f[1600]), so `verify(raw, modified_content)` returns `false`.

B Quick Reference

Property	Value
Wire size	73 bytes
Header size	9 bytes
Tail size	64 bytes (512 bits)
Tail primitive	SHAKE-256 (FIPS 202)
Timestamp field	48-bit big-endian ms, offset 0
Variant byte	`0x03` at offset 6
Counter field	16-bit big-endian, offset 7
Encoding length: Base64URL	98 chars, no padding
Encoding length: Crockford	117 chars, case-insensitive on decode
Classical collision bound	2^{256}
Quantum collision bound (BHT)	$\sim 2^{170}$
Classical second-preimage	$\min(2^d, 2^{256}) = 2^{256}$ (FIPS 202 cap)
Quantum second-preimage (Grover)	2^{256}
Timestamp field range	$[0, 2^{48} - 1]$ ms (1970 – ≈10889)
Min sanity timestamp (non-normative)	1,700,000,000,000 ms ≈ 2023-11-14 (TesseraID-parity)

C Glossary

BHT (Brassard–Høyer–Tapp). Quantum collision-finding algorithm (Brassard et al., 1998). Costs $\sim 2^{n/3}$ operations for an n-bit space.

Birthday bound. Classically $\sqrt{\pi/2} \cdot 2^{n/2}$ generations before an expected collision in an n-bit random space.

Content identifier. An identifier derived deterministically from content (TesseraHash, IPFS CID, Git commit hash) as opposed to a random unique identifier (TesseraID, UUIDv4, ULID).

Crockford Base32. Douglas Crockford's Base32 alphabet (`0-9` then `A-Z` minus `I/L/O/U`) (Crockford). Case-insensitive on decode; ASCII-sort-preserving.

Grover's algorithm. Quantum search reducing $O(N)$ to $O(\sqrt{N})$ (Grover, 1996).

HMAC. Keyed hash construction (Krawczyk et al., 1997). TesseraHash is not an HMAC; see Section 6.

KAT (Known-Answer Test). A test vector pinning an exact expected output for a given input. TesseraHash's KAT vectors live in `conformance/vectors/`.

Keccak-f[1600]. The permutation underlying SHA-3 and SHAKE (fip, 2015).

SHAKE-256. The variable-output-length member of the SHA-3 family (fip, 2015), with a 256-bit security level. TesseraHash uses it with a fixed 64-byte (512-bit) output.

Variant byte. Byte at offset 6 of a TesseraHash. Always `0x03` for v1.

D The Tessera Monorepo

For the cross-project relationship (variant-byte allocation across TesseraID/TesseraHash/TesseraSeal, the canonical TesseraID-as-AD and TesseraHash-as-AD patterns, shared `common/` library code, shared conformance harness), see Section 9. The summary of cross-project byte allocations:

Byte	Owner	Meaning	Status
`0x00`	—	Permanently reserved (never assign)	MUST reject
`0x01`	TesseraID	Standard generator	assigned
`0x02`	TesseraID	HLC generator	assigned
`0x03`	TesseraHash	SHAKE-256 / 64-byte tail	assigned (this document)
`0x04`	TesseraSeal	Encryption envelope (suite byte at offset 7)	assigned
`0x05`–`0xFE`	—	Reserved-future	MUST reject
`0xFF`	—	Permanently reserved (never assign)	MUST reject

E Author and Document Provenance

Author: Bahram Samani, Tessera monorepo maintainer.

This document is built from LaTeX sources under `tessera-hash/latex/` (`main.tex` + `secNN_*.tex`); the build chain is `pdflatex` → `bibtex` → `pdflatex` × 2 per the Makefile.

Authority order (when sources disagree): this whitepaper, then the Python reference at `tessera-hash/python/tes` then the canonical C ABI at `tessera-hash/c/tessera_hash.h`, then every other language. A cross-language disagreement is never resolved by changing the spec to match an outlier; the outlier is fixed.

Hard invariants (from `tessera-hash/CLAUDE.md`):

- **Hash primitive: SHAKE-256.** Keccak-f[1600] sponge with arbitrary output length, used here with fixed 64-byte (512-bit) output.
- **Vendored Keccak.** C/C++/Java implementations use the shared Keccak source under `common/`; no project re-vendors a second copy.
- **Canonical encodings.** Base64URL (98 chars, no padding) and Crockford Base32 (117 chars, case-insensitive on decode, `I/L` → `1` and `O` → `0` confusables accepted on decode). Encoded lengths are fixed per variant.
- **Variant discipline.** `0x03` only; any unknown variant byte is `TESSERA_HASH_E_UNKNOWN_VAR` — enforced by the binary decoder and by both text decoders at decode time. Cross-project bytes (`0x01`, `0x02`, `0x04`) and reserved-future bytes are rejected.
- **Generator locking.** The generator is internally locked where the language has shared-memory threading; the `(last_ts, counter)` read-modify-write is atomic. The C struct is caller-synchronized by documented contract (`tessera_hash.h`: declare one per thread or wrap with a mutex); the C++ wrapper inherits that contract.

Conformance gate: `tools/conformance/run.sh` runs on every pull request and push, and its cross-language job is a required branch-protection check on `main`; TesseraID, TesseraHash, and

TesseraSeal flow through the same harness. TesseraHash's KAT count: 16 `decode`, 8 `hash_only`, encode/decode round-trip pairs for both encodings, `verify` positive/negative cases, 24 `invalid` negative cases (including text-form wrong-variant records). Latest cross-language run (2026-06-13) is green in all eight languages across the full three-project vector set: Python/Go/Rust/C 297 PASS each (C reached parity once its TesseraSeal hybrid `0x06` port landed, TSEAL-00030), C++/TypeScript/Java 229 PASS, CPython ext 224 PASS + 11 SKIP (the non-seal languages skip the TesseraSeal vector directory).

References

SHA-3 standard: Permutation-based hash and extendable-output functions. Technical Report FIPS PUB 202, National Institute of Standards and Technology, Aug. 2015. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.202.pdf.

Module-lattice-based key-encapsulation mechanism standard. Technical Report FIPS PUB 203, National Institute of Standards and Technology, Aug. 2024a. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.pdf.

Module-lattice-based digital signature standard. Technical Report FIPS PUB 204, National Institute of Standards and Technology, Aug. 2024b. URL https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.pdf.

A. Biryukov, D. Dinu, D. Khovratovich, and S. Josefsson. The memory-hard Argon2 password hash and proof-of-work function. RFC 9106, IETF, 2021. URL https://www.rfc-editor.org/rfc/rfc9106.

S. Bradner. Key words for use in RFCs to indicate requirement levels. BCP 14, RFC 2119, IETF, 1997. URL https://www.rfc-editor.org/rfc/rfc2119.

G. Brassard, P. Høyer, and A. Tapp. Quantum cryptanalysis of hash and claw-free functions. In *LATIN '98*, 1998. URL https://arxiv.org/abs/quant-ph/9705002.

D. Crockford. Base32. URL https://www.crockford.com/base32.html.

A. Feerasta. ULID: Universally unique lexicographically sortable identifier, 2016. URL https://github.com/ulid/spec.

L. K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996. URL https://arxiv.org/abs/quant-ph/9605043.

S. Josefsson. The Base16, Base32, and Base64 data encodings. RFC 4648, IETF, 2006. URL https://www.rfc-editor.org/rfc/rfc4648.

S. Josefsson and I. Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, IETF, 2017. URL https://www.rfc-editor.org/rfc/rfc8032.

H. Krawczyk, M. Bellare, and R. Canetti. HMAC: Keyed-hashing for message authentication. RFC 2104, IETF, 1997. URL https://www.rfc-editor.org/rfc/rfc2104.

B. Leiba. Ambiguity of uppercase vs lowercase in RFC 2119 key words. BCP 14, RFC 8174, IETF, 2017. URL https://www.rfc-editor.org/rfc/rfc8174.

Paragon Initiative Enterprises. PASETO: Platform-agnostic security tokens, v4. URL https://github.com/paseto-standard/paseto-spec.